

Banking System:

Complete Banking System – Technical Design Document

1. 📌 Executive Summary

This document outlines the architecture, components, and implementation strategy for a full-featured banking system. It supports core banking operations such as customer onboarding, account management, transactions, loans, compliance, and reporting. The system is designed to be secure, scalable, and compliant with financial regulations.

2. 👥 User Roles & Access Control

Role	Capabilities
------	--------------

Customer	View balances, transfer funds, apply for loans, manage cards and accounts
----------	---

Bank Staff	Approve transactions, manage customer profiles, handle support
------------	--

Admin	Configure system settings, manage roles, audit logs
-------	---

Auditor	Access reports, monitor compliance, view logs
---------	---

Access Control:

- Role-Based Access Control (RBAC)
- Fine-grained permissions per module
- Multi-Factor Authentication (MFA) for sensitive operations

3. 🏗️ System Architecture

Architecture Type: Microservices

Core Layers:

- **Presentation Layer:** Web & Mobile apps
- **Service Layer:** RESTful APIs for business logic
- **Data Layer:** Relational DB for transactions, NoSQL for logs/cache
- **Integration Layer:** External services (KYC, payments, AML)

Key Microservices:

- Customer Service

- Account Service
- Transaction Service
- Loan Service
- Card Service
- Notification Service
- Authentication Service
- Audit & Compliance Service

Communication:

- Internal: gRPC or REST
- External: REST APIs with OAuth 2.0

Message Queue: Kafka for asynchronous operations (e.g., notifications, audit logs)

4. Frontend Design

Web App (Bank Staff/Admin)

- Built with React.js or Angular
- Role-based dashboards
- Real-time transaction monitoring
- Audit trail viewer

Mobile App (Customer)

- Built with Flutter or React Native
- Secure login with biometric support
- Account overview, fund transfers
- Loan application and tracking

Security:

- TLS encryption
- Secure storage of tokens
- Device fingerprinting

5. Backend Services

Authentication Service

- OAuth 2.0 / OpenID Connect
- JWT tokens with refresh mechanism
- MFA support via SMS/email/Authenticator apps

Customer Service

- Onboarding with KYC verification
- Profile management
- Risk scoring

Account Service

- Create/check savings/current accounts
- Link cards and loans
- Freeze/unfreeze accounts

Transaction Service

- Internal transfers (IMPS/NEFT/RTGS)
- External payments via UPI/SWIFT
- Reconciliation and rollback mechanisms

Loan Service

- Loan application and approval workflow
- EMI calculation and scheduling
- Delinquency tracking

Notification Service

- SMS, email, push notifications
- Templates for alerts, OTPs, reminders

6. Database Design

Relational DB (PostgreSQL / Oracle)

Schema Highlights:

- Customers: Personal info, KYC status
- Accounts: Account type, balance, status
- Transactions: Amount, source, destination, timestamp
- Loans: Principal, interest, EMI schedule
- Cards: Type, limits, status
- AuditLogs: Action, actor, timestamp

NoSQL (MongoDB / Redis)

- Caching account balances
- Storing notification queues
- Session management

Data Integrity:

- ACID compliance for financial operations
- Foreign key constraints
- Triggers for audit logging

7. 🚀 API Design

RESTful APIs (OpenAPI Spec)

Examples:

- POST /api/v1/customers – Create customer
- GET /api/v1/accounts/{id} – Fetch account details
- POST /api/v1/transactions – Initiate transfer
- GET /api/v1/loans/{id} – Loan status
- POST /api/v1/auth/login – Authenticate user

Best Practices:

- Pagination for list endpoints
- Rate limiting and throttling

- API versioning
- Input validation and sanitization

8. Security & Compliance

Security Measures

- AES-256 encryption for data at rest
- TLS 1.3 for data in transit
- Secure password hashing (bcrypt)
- Role-based access control
- Session timeout and auto-logout

Compliance

- GDPR for data privacy
- PCI-DSS for card handling
- RBI/FDIC/SEBI guidelines for banking operations
- Audit logs for every sensitive action
- Data retention policies

9. Integrations

Payment Gateways

- NPCI (UPI, IMPS, NEFT, RTGS)
- SWIFT for international transfers
- Stripe/Razorpay for card payments

KYC & AML

- Digilocker, Onfido, Jumio for identity verification
- AML screening via World-Check or Dow Jones Watchlist

Credit Scoring

- Integration with CIBIL/Experian APIs
- Real-time score fetching during loan application

10. 📈 Scalability & Performance

Infrastructure

- Kubernetes for container orchestration
- Horizontal scaling of stateless services
- Redis for caching frequently accessed data
- CDN for static assets

Performance Optimization

- Connection pooling
- Query optimization and indexing
- Async processing for non-critical tasks
- Load balancing with HAProxy/NGINX

11. 🧪 Testing Strategy

Type	Tools Used	Scope
Unit Testing	JUnit / Jest	Service logic
Integration Tests	Postman / RestAssured	API endpoints
E2E Testing	Cypress / Selenium	Full user workflows
Load Testing	JMeter / Locust	Transaction throughput, latency
Security Testing	OWASP ZAP / Burp	Vulnerability scanning

12. 🚀 Deployment Strategy

CI/CD Pipeline

- GitHub Actions / Jenkins
- Dockerized services
- Helm charts for Kubernetes deployment
- Canary releases for critical services

Monitoring & Logging

- Prometheus + Grafana for metrics
- ELK stack for logs
- Alerting via PagerDuty / Opsgenie

13. Reporting & Analytics

Dashboards

- Transaction volume and trends
- Loan performance and delinquency
- Customer segmentation
- Regulatory compliance reports

Tools

- Metabase / Power BI / Tableau
- Scheduled exports to regulators
- Real-time fraud detection alerts